Loc-Camel-Route

Alberto Espinosa (KS-Group)

Oct 19th, 2025

Abstract

This document describes a contract-first, BPEL-driven integration sample and its transformation into a Spring + Maven + Camel implementation using local agentic tooling. It clarifies the problem, the original technology stack and artefacts, the proposed agent architecture and information flow, and the expected results. It also defines project organisation and installation scripts required to run the pipeline without mocks, using only real endpoints and data.

1 Problem Statement

The sample contains SOAP (Simple Object Access Protocol: XML-based messaging protocol for web services) WSDL (Web Services Description Language: XML format describing service interfaces)/XSD (XML Schema Definition: XML format defining data structures) contracts and a BPEL (Business Process Execution Language: XML language for orchestrating web-service workflows) process *(LocationRetrievalLOC3X1Process)* that orchestrates two inbound operations and multiple partner service calls. There is no runnable Java build, no proper server host, and legacy Java snippets are considered non-authoritative. The problem is to translate this into a robust Spring Boot (Java micro-service framework) + Maven (Java build/dependency tool) project with Apache Camel (integration routing engine)/CXF (Java web-service framework), regenerate models and stubs from contracts, implement deterministic mappings that replace BPEL assignments, externalise configuration, and provide contract and integration verification. The transformation must be automated by local agents and verified end-to-end without mocks; integration results depend on live endpoints.

2 Original Technology and Artefacts

2.1 Overview

The original sample is contract-first and IBM BPEL-centric:

- WSDL/XSD defining document/literal SOAP operations for Location and Country services.
- BPEL process (IBM WebSphere style) with partner links, variables, fault handlers, and embedded Java logging/fault composition.
- HTTP binding WSDLs referencing localhost endpoints.
- Shared fault schema (SimpleFault.xsd).

2.2 Organisation Diagram

Figure 1 illustrates the folder organisation of the original sample.

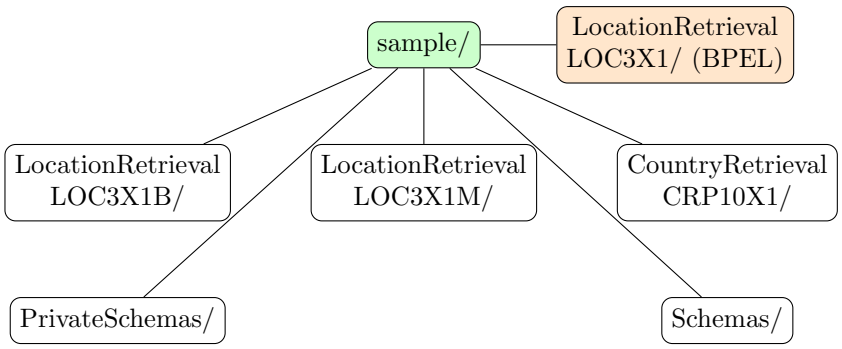

Figure 1: Folder organisation of the original sample.

3 File Structure and Standards

3.1 Summary Table

Table 1 summarises the key folders and files, their types, and roles.

Table 1: File structure summary of the original sample.

Folder/File	**Type**	**Role**
LocationRetrieval LOC3X1 / LocationRetrieval LOC3X1Process.bpel	BPEL	Orchestration: pick/onMessage for inbound operations, partner links (LOC3X1M, CRP11X1, CRP10X1), variables, fault handlers, logging.
LocationRetrieval LOC3X1B / LocationRetrieval-LOC3X1B.wsdl	WSDL	Public interface: LOC3X1B operations, request/response messages, faults.
LocationRetrieval LOC3X1M / LocationRetrieval-LOC3X1M.wsdl	WSDL	Internal interface: LOC3X1M operations to which requests are transformed.
CountryRetrieval CRP10X1 / CountryRetrieval-CRP10X1.wsdl	WSDL	Partner interface: country retrieval operation(s).
PrivateSchemas / SimpleFault.xsd	XSD	Shared fault structure (FaultMessageText).
Schemas/*	XSD	Domain types referenced by requests/replies across services.

File types: functions, protocols, and standards

- **BPEL (Business Process Execution Language)**: Defines executable orchestration of service interactions (picks, onMessage, invoke, assign, fault handlers). It runs on a BPEL engine and coordinates partner links and variables to implement business processes, typically calling SOAP-based services described by WSDL. Common deployment uses HTTP(S) transport and adheres to vendor-specific extensions (e.g., IBM WebSphere styles) while following WS-BPEL principles.

- **WSDL (Web Services Description Language)**: Specifies service contracts, including operations (portTypes), request/response messages, bindings, and service endpoints. In this sample WSDLs are document/literal SOAP with HTTP(S) bindings and include fault definitions. Good practice is alignment with WS-I Basic Profile and consistent namespaces and versions for cross-service compatibility.
- **XSD (XML Schema Definition)**: Provides formal XML type definitions for messages and domain objects used by WSDL services and BPEL variables. XSDs enforce structure and validation, carry namespaces for modularity, and support versioning. The shared SimpleFault.xsd standardises fault payloads across services, enabling consistent error reporting.

4 Project Description

4.1 Objectives

Create an automated, agent-driven translation from BPEL + contracts into a Spring Boot + Maven project using Camel/CXF, with deterministic mappings and centralised fault handling, verified via contract checks and real integration tests.

4.2 Feasibility and Importance

Feasibility is high because the assets are contract-first and the BPEL encodes mediation steps. Importance lies in replacing legacy BPEL with modern Java micro-integration while preserving behaviour, improving maintainability, and enabling reproducible automation.

4.3 Challenges and Fine Points

IBM-specific BPEL features (BOFactory, SDO) must be mapped to plain JAXB + Spring. Some imports may be missing, and business rule nodes (e.g., fire district checks) require explicit specifications. End-to-end tests depend on live partner endpoints; when unavailable, tests must be reported as blocked, never simulated.

4.4 Used Technology

- **Deterministic agents via shell/Python; optional LLM reviewer (Ollama/LangGraph)** — Reproducible, CI-safe automation core with an optional, explainability-only reviewer.
- **Java 17** — Long-term-support JVM release providing modern language features and performance.
- **Maven** — Declarative Java build and dependency-management tool.
- **Spring Boot** — Opinionated micro-service framework for rapid, production-ready Spring applications.
- **Apache Camel** — Enterprise-integration patterns engine for declarative routing and mediation.
- **Apache CXF** — Open-source services framework supporting JAX-WS/JAX-RS and SOAP/REST bindings.
- **JAXB/JAX-WS tooling** — Standard XML binding and web-service stub generation from WSDL/XSD.

- **MapStruct** — Compile-time code generator for type-safe, high-performance object mappings.
- **SLF4J/Logback** — Simple logging façade with fast, configurable log implementation.
- **JUnit** — Ubiquitous testing framework for unit and integration verification.
- **Shell/Python automation drivers** — Portable scripts executed inside the project virtual environment to drive agent workflows.
- **jq/yq** — CLI tools for JSON/YAML processing used by contract discovery and preflight reporting.
- **Docker Docker Compose** — Containerisation of services, clients, orchestration, FastAPI, and optional DB; local multi-service orchestration.
- **CICD (GitHub Actions or Jenkins)** — Deterministic pipeline executing preflight, build, test, packaging, and container publish.
- **Node.js React** — Frontend dashboard for evidence browsing and orchestration controls.
- **Python FastAPI** — Lightweight API for evidence metadata and agent orchestration hooks.

4.5 Local Models via Ollama

- Reviewer/Interpreter model (optional): runs after evaluation to generate an executive explanation report from artefacts; never changes code or gates release.
- Deterministic core: planning, scaffolding, build, tests, evaluation executed by shell/Python without LLM; reproducible and CI-safe.
- Retrieval (optional): embeddings used solely to fetch relevant artefact snippets for the reviewer; not required for the core.

Table 2 details the optional reviewer model and supporting components when enabled.

Table 2: Recommended local models and rationale.

Model Role	**Recommended Model(s)**	**Rationale**
Instruction / Planner	Llama 3 Instruct (local)	Strong instruction-following aids planning, decomposition, and policy adherence (no mocks, British English), producing reliable agent steps and acceptance criteria.
Code Generation	CodeLlama or Qwen Coder (local)	Trained for programming tasks; produces idiomatic Java, Spring Boot, Maven, Camel, and CXF code with fewer syntax errors and better library usage patterns.

Embedding / Retrieval	nomic-embed-text or mxbai-embed-large (local)	High-quality embeddings for contract/code search, enabling deterministic mapping between WSDL/XSD schemas and generated classes, improving route wiring accuracy.

4.6 Agentic System Architecture

Agents: Planner, Contract Reader, BPEL Parser, Schema Aligner, Integration Builder (CXF/Camel), Mapping Builder (MapStruct), Test Designer, Environment Installer, Evaluator/Referee. Each agent has a focused remit and produces artefacts consumed downstream.

The Environment Installer performs a tooling preflight, verifies JDK/Maven and auxiliary tools (jq/yq, Python packages), installs missing dependencies where permitted, and emits a versioned readiness report at `doc/tooling-report.md`. It ensures all downstream agents can compile, run, capture logs, and evaluate correctness.

Figure 2 presents the agentic system pipeline, congruent with Table 3. Figure 3 depicts the high-level mediation flow derived from BPEL.

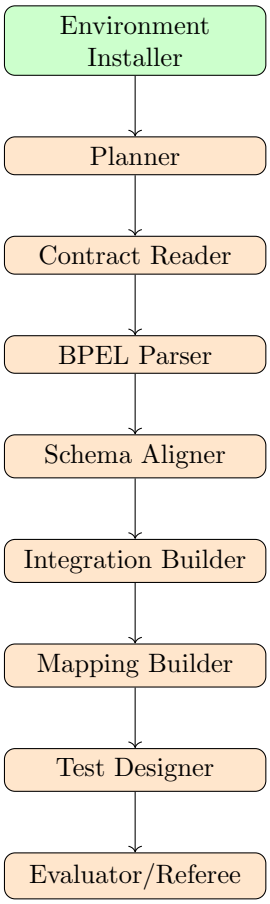

Figure 2: Agentic system pipeline congruent with the summary table.

4.7 Expected Results

A runnable inbound BasicService (Spring Boot + CXF) with SOAP Ping; Camel routes for LOC3X1B consuming via CXF PAYLOAD; outbound partner endpoints configured as CXF

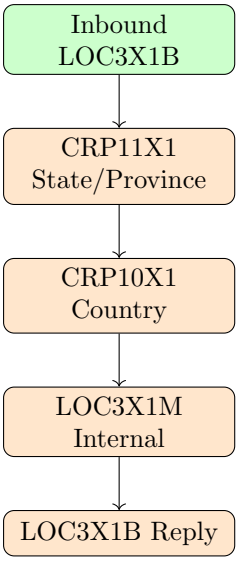

Figure 3: High-level flow derived from BPEL mediation.

PAYLOAD clients; typed mappings implemented where available with stubs elsewhere; centralised error handling using a placeholder SimpleFault (JSON body) with plans to map to schema; contract code generation available; integration tests capture evidence and mark external calls blocked without live endpoints.

Note: The orchestration specification includes a `data_mappings` *section derived deterministically by parsing original IBM* `.map` *XML artefacts (e.g.,* `map:businessObjectMap` *and* `map:propertyMap`*). This reflects IBM BPEL script-based transformations rather than* `bpws:assign` *elements.*

5 Project Organisation

Recommended directory layout (generalised monorepo):

```
project-root/
|-- doc/                # LaTeX, tickets, plans, reports
|-- scripts/            # Automation (preflight, build, run, tests, test_master)
|-- contracts/          # Inventories, canonical maps, generated artefacts
|-- modules/            # Maven multi-module: contracts, inbound, clients/{loc3x1m,crp11x1,c
|-- docker/             # Dockerfiles and docker-compose.yml
|-- server/             # FastAPI evidence API (pipeline runs, logs, summaries)
`-- tests/              # Test artefacts and JUnit XML outputs (mirrors sections)
```

6 Installation Scripts (Definition)

- `scripts/preflight.sh` – Runs tooling preflight, verifies JDK/Maven and CLI tools, produces `doc/tooling-report.md`, and attempts a smoke build if a Maven project is present.
- `scripts/bootstrap_java.sh` – Installs JDK and Maven locally so the build toolchain is ready.
- `scripts/generate_contracts.sh` – Runs JAXB/CXF codegen to create Java models and service stubs from WSDL/XSD.
- `scripts/build_all.sh` – Executes Maven compile, test and package for the entire project.

Table 3: Agent responsibilities, inputs, and outputs.

Agent	Responsibility	Inputs	Outputs
Planner	Global plan, dependency ordering, acceptance criteria alignment	BPEL, WSDL/XSD inventory	Execution plan, module breakdown
Contract Reader	Generate JAXB/CXF artefacts and validate contracts	WSDL/XSD	Models, stubs, contract validation report
BPEL Parser	Extract inbound ops, partner invocations, variables, fault logic	BPEL	Orchestration spec, mapping intents
Schema Aligner	Namespace/version governance and schema catalogue curation	XSD set	Canonical schema map, alignment report
Integration Builder	Configure CXF endpoints and Camel routes	Orchestration spec, properties	Running inbound service, route definitions
Mapping Builder	Deterministic request/response transformations	Orchestration spec, models	Mapper implementations and tests
Environment Installer	Provision local JDK/Maven and project tooling	Script definitions	Installed toolchain, verified versions
Evaluator/Referee	Verify builds, runtime, tests; produce final audit	Build outputs, test reports	Evaluation summary, blockers list

- `scripts/run_services.sh` – Starts inbound BasicService (CXF '/services/BasicService'), runs SOAP Ping, and captures evidence.
- `scripts/test_integrations.sh` – Runs integration tests against live partner endpoints (no mocks).
- `scripts/test_master.sh` – Master test runner to execute unit tests by ordinal (LOC-XXX), by section, or all.

All scripts run inside the project virtual environment for consistent orchestration.

7 Agentic Strategy and Implementation Philosophy

7.1 Overview of the Agentic Approach

The transformation from BPEL-based integration to modern Spring Boot + Camel architecture employs an *agentic strategy* that fundamentally differs from traditional code generation or manual migration approaches. Rather than producing static artefacts that become obsolete when contracts change, this system implements a **closed-loop, evidence-driven integration layer** that continuously adapts to evolving service landscapes whilst maintaining operational stability and auditability.

The core philosophy centres on **dynamic resilience over static perfection**. The agentic system prioritises maintaining functional integrations even when partner services change locations, bindings, or schemas, rather than requiring manual intervention for every contract modification. This approach recognises that enterprise integration environments are inherently volatile, with services frequently updated, relocated, or temporarily unavailable.

7.2 The Closed-Loop Integration Cycle

The agentic system operates through a continuous seven-phase cycle: **Discover → Validate → Synthesise → Verify → Harden → Monitor → Adapt**. This cycle ensures that integration behaviour remains consistent whilst the underlying implementation adapts to environmental changes.

Figure 4 illustrates this closed-loop process, showing how each phase feeds into the next whilst maintaining evidence trails for governance and auditability.

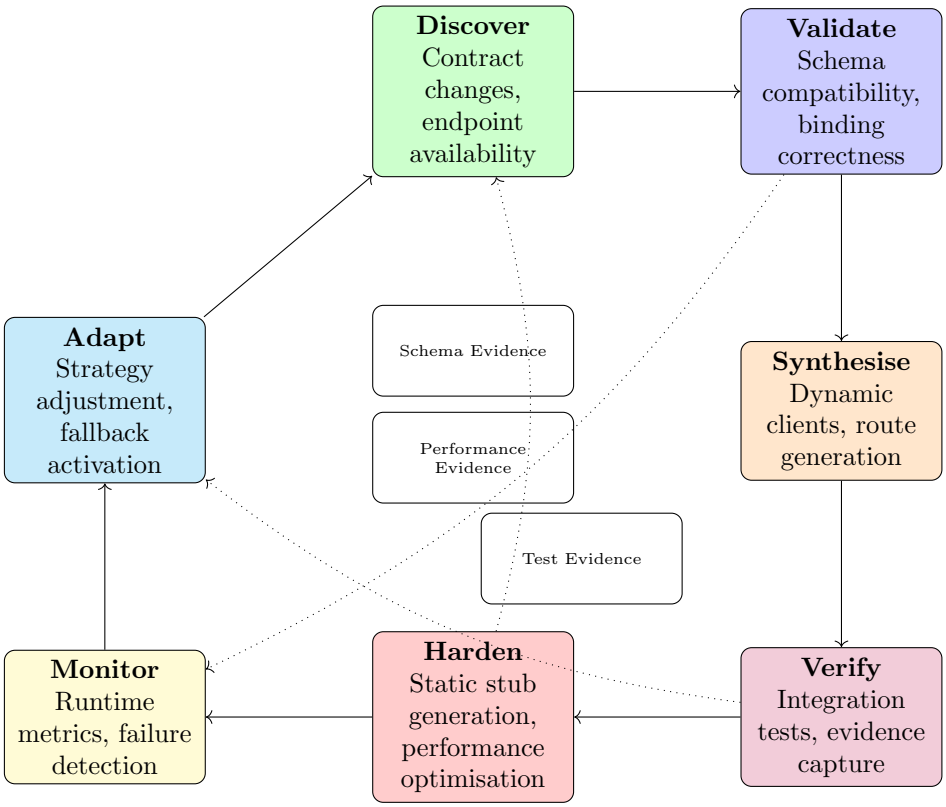

Figure 4: The closed-loop agentic integration cycle with evidence flows.

7.2.1 Phase Descriptions

- **Discover**: Continuously scans for changes in WSDL locations, schema versions, endpoint availability, and BPEL process definitions. Uses configurable polling intervals and change detection algorithms.
- **Validate**: Performs schema compatibility checks, binding validation, and import resolution. Identifies breaking changes versus compatible extensions.
- **Synthesise**: Generates integration artefacts using either static code generation (for stable contracts) or dynamic invocation patterns (for volatile contracts). Chooses strategy based on contract stability metrics.
- **Verify**: Executes targeted integration tests against live endpoints, capturing evidence of success or failure. Never uses mocks—blocked tests are reported as such.
- **Harden**: Converts proven dynamic integrations into optimised static implementations when stability thresholds are met. Maintains both versions during transition periods.

- **Monitor**: Tracks runtime performance, error rates, and availability metrics. Detects degradation patterns that trigger cycle re-entry.
- **Adapt**: Adjusts integration strategies based on accumulated evidence. May switch from static to dynamic approaches or activate fallback mechanisms.

7.2.2 Deterministic Core and Reviewer Agent

The core pipeline is **deterministic**, repository-aware, and evidence-driven. Planning, scaffolding, build, tests, and evaluation are performed by guarded shell/Python scripts that operate only on explicit artefacts (WSDL/XSD, BPEL, YAML specs), enforce virtual-environment execution, and persist logs and run metadata. This yields reproducibility, auditability, and CI compatibility; no local LLM is required for correctness.

Reviewer/Interpreter (Optional) When enabled, a local LLM runs *after* evaluation to interpret artefacts and produce an executive explanation report. Inputs include scaffold/build logs, test results, the agent plan, and route specifications. Outputs are written to ‘doc/reviewer-report.md‘ as a plain-English narrative of decisions, evidence chains, and risks. The reviewer never changes code or gates release; it augments explainability only.

Guardrails LLM outputs are schema-validated and cross-checked against deterministic artefacts; disagreements are flagged, not auto-applied. The deterministic core remains authoritative regardless of reviewer presence.

7.3 Static versus Dynamic Invocation Strategy

The agentic system employs a **hybrid invocation model** that dynamically selects between static code generation and runtime service discovery based on contract stability and operational requirements.

Figure 5 depicts the decision matrix used to determine the optimal invocation approach for each service integration.

7.3.1 Static Generation Benefits

- **Performance**: Compile-time optimisation eliminates runtime reflection overhead
- **Type Safety**: Strong typing catches integration errors at build time
- **IDE Support**: Full code completion and refactoring capabilities
- **Debugging**: Stack traces reference actual Java classes rather than dynamic proxies

7.3.2 Dynamic Invocation Benefits

- **Adaptability**: Continues functioning when contracts change without recompilation
- **Resilience**: Graceful degradation when endpoints become temporarily unavailable
- **Discovery**: Automatic detection of new operations or modified schemas
- **Fallback**: Can switch between multiple endpoint versions transparently

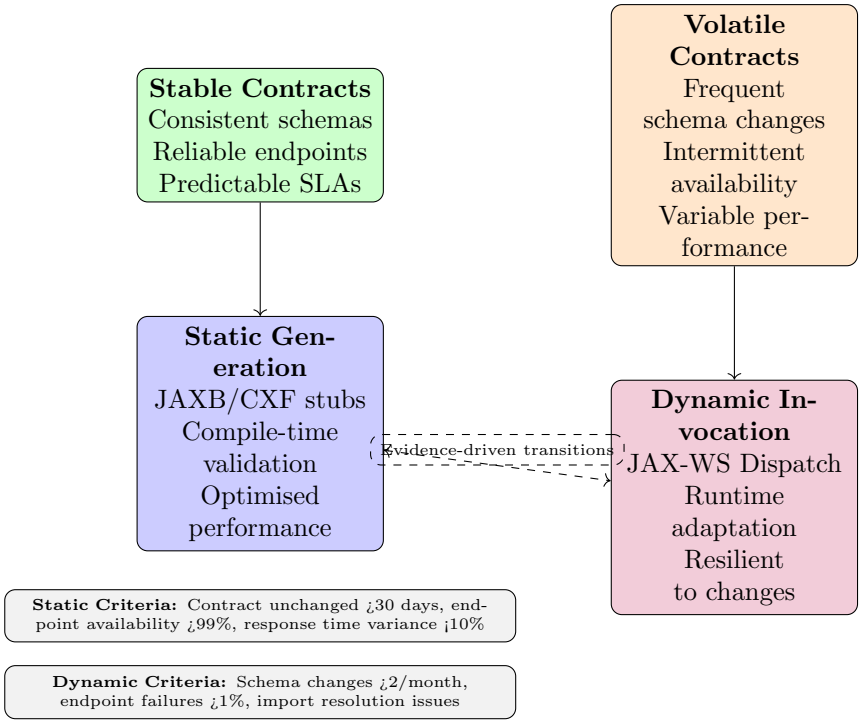

Figure 5: Static versus dynamic invocation decision matrix.

7.4 Evidence-Driven Hardening Process

The transition from dynamic to static implementations follows a rigorous **evidence-driven hardening process** that ensures stability before committing to optimised static code generation.

7.4.1 Evidence Collection Metrics

- **Contract Stability**: Time since last schema change, frequency of endpoint modifications
- **Operational Reliability**: Success rate, response time consistency, availability percentage
- **Integration Complexity**: Number of transformations, fault handling requirements, orchestration depth
- **Business Criticality**: SLA requirements, transaction volume, downstream dependencies

7.4.2 Hardening Thresholds

Static generation is triggered when all of the following criteria are met for a continuous period:

- Contract unchanged for minimum 30 days
- Endpoint availability exceeding 99.5%
- Response time variance below 10% of baseline
- Zero import resolution failures
- Successful integration test rate above 98%

7.5 Orchestration Synthesis from BPEL

The agentic system transforms BPEL orchestration logic into Camel routes through a sophisticated **orchestration synthesis process** that preserves business logic whilst modernising the execution environment.

Figure 6 illustrates how BPEL constructs are mapped to equivalent Camel patterns whilst maintaining the original orchestration semantics.

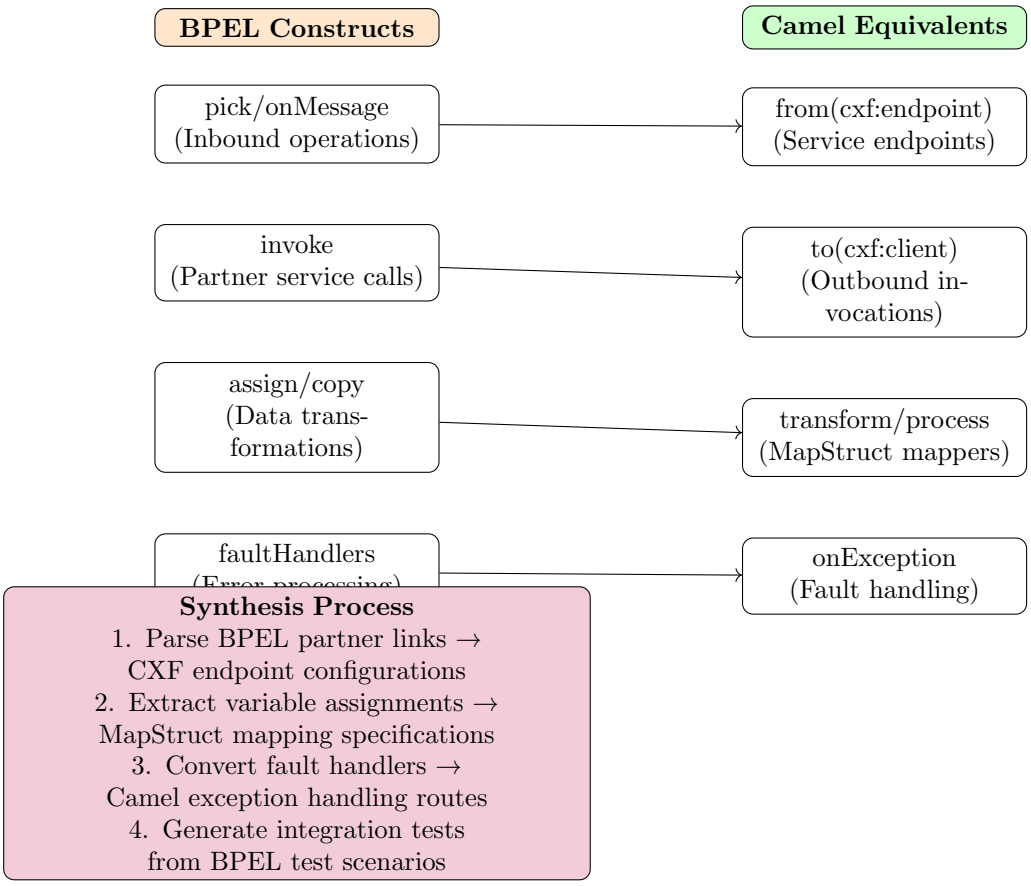

Figure 6: BPEL to Camel orchestration synthesis mapping.

7.5.1 Synthesis Algorithms

- **Partner Link Analysis**: Extracts service dependencies and generates corresponding CXF client configurations with appropriate timeout and retry policies
- **Variable Flow Tracking**: Maps BPEL variable assignments to type-safe MapStruct transformations, preserving data lineage and validation rules
- **Fault Propagation**: Converts BPEL fault handlers into Camel exception handling with appropriate compensation and rollback logic
- **Correlation Preservation**: Maintains message correlation patterns from BPEL correlation sets using Camel's correlation identifier mechanisms

7.6 Governance and Safety Mechanisms

The agentic system incorporates comprehensive **governance and safety mechanisms** to ensure that automated decisions remain auditable, reversible, and aligned with operational

policies.

7.6.1 Evidence Gates

Every significant system change must pass through **evidence gates** that require documented justification and approval criteria:

- **Contract Changes**: Schema compatibility analysis, impact assessment, rollback procedures
- **Endpoint Modifications**: Availability verification, performance benchmarking, fallback validation
- **Route Alterations**: Integration test results, business logic preservation, monitoring coverage
- **Hardening Decisions**: Stability evidence, performance improvements, maintenance implications

7.6.2 Auditability Requirements

All agentic decisions are logged with sufficient detail for post-hoc analysis:

- **Decision Context**: Environmental conditions, available evidence, applied policies
- **Alternative Considerations**: Other strategies evaluated, rejection rationale
- **Implementation Details**: Generated artefacts, configuration changes, test results
- **Success Metrics**: Performance improvements, reliability gains, maintenance reductions

7.6.3 Safety Constraints

The system operates within strict safety boundaries to prevent operational disruption:

- **No Secret Exposure**: Credentials and sensitive configuration never logged or committed
- **Gradual Rollout**: Changes deployed incrementally with monitoring at each stage
- **Automatic Rollback**: Failed changes trigger immediate reversion to last known good state
- **Human Override**: Manual intervention capabilities for emergency situations

7.7 Production Operational Model

In production environments, the agentic system operates as a **self-healing integration layer** that maintains service continuity whilst adapting to changing conditions.

7.7.1 Continuous Monitoring

- **Contract Drift Detection**: Automated scanning for WSDL/XSD changes with impact analysis
- **Performance Baseline Tracking**: Statistical analysis of response times, throughput, and error rates
- **Availability Monitoring**: Endpoint health checks with intelligent retry and circuit breaker patterns

- **Integration Quality Metrics**: Success rates, data quality scores, business rule compliance

7.7.2 Adaptive Response Strategies

When environmental changes are detected, the system employs graduated response strategies:

1. **Transparent Adaptation**: Minor changes handled automatically without service interruption
2. **Graceful Degradation**: Temporary fallbacks activated whilst permanent solutions are synthesised
3. **Controlled Failover**: Systematic switching to alternative endpoints or service versions
4. **Emergency Isolation**: Problematic integrations quarantined to prevent cascade failures

7.7.3 Expected Production Outcomes

The agentic approach delivers several key operational benefits:

- **Reduced Maintenance Overhead**: Automatic adaptation eliminates manual intervention for routine changes
- **Improved Reliability**: Multiple fallback strategies and proactive monitoring prevent service disruptions
- **Enhanced Visibility**: Comprehensive evidence trails enable rapid problem diagnosis and resolution
- **Accelerated Integration**: New services can be integrated rapidly using proven patterns and automated testing

This agentic strategy represents a fundamental shift from reactive maintenance to proactive adaptation, enabling integration architectures that evolve gracefully with their operational environments whilst maintaining the reliability and auditability required for enterprise systems.

8 End-to-End Pseudocode Workflow

8.1 Guide: Purpose, Inputs, BPEL, Translation, and Validation

Purpose in plain terms This project takes an **IBM-style BPEL** orchestration and turns it into a **modern Java implementation** using Spring Boot, Maven, Apache Camel, Apache CXF, and MapStruct. In human terms: we read the original service contracts (WSDL/XSD) and the orchestration logic (BPEL), then generate Java code and routes that behave like the original process, with tests and evidence so you can trust what runs locally.

What is the input? The input is the `sample/` folder containing:

- BPEL: `sample/LocationRetrievalLOC3X1/LocationRetrievalLOC3X1Process.bpel`
- Public WSDL: `sample/LocationRetrievalLOC3X1B/LocationRetrievalLOC3X1B.wsdl`
- Internal/Partner WSDLs: `sample/LocationRetrievalLOC3X1M/LocationRetrievalLOC3X1M.wsdl`, `sample/CountryRetrievalCRP10X1/CountryRetrievalCRP10X1.wsdl`, `sample/StateOrProvinceRetri`
- Shared XSDs: `sample/Schemas/*` and faults in `sample/PrivateSchemas/SimpleFault.xsd`

How we dissect it: we list all WSDL/XSDs (contract inventory), parse the BPEL to extract the inbound operations, partner calls and fault paths, and record types and namespaces referenced by messages.

Essential BPEL concepts with a micro-example BPEL is an **orchestration language** that describes how a service should *receive* a message, *transform* parts of it, *invoke* other services, and *return* a reply or a fault. Below is a **5-line** excerpt from the real LOC3X1 BPEL file (trimmed for clarity) followed by the matching Java/Camel translation so you can see the same intent in both worlds.

```
<!-- 1. BPEL fragment -->
<receive name="receiveLOC3X1B"
         partnerLink="LOC3X1B"
         operation="getLocation"
         variable="locRequest" />
<invoke   name="callCRP11X1"
         partnerLink="CRP11X1"
         operation="getState"
         inputVariable="stateReq"
         outputVariable="stateResp" />
```

```
// 2. Camel route fragment
from("cxf:/LOC3X1B?serviceClass=LOC3X1B_Service")
  .to("direct:getLocation")
from("direct:getLocation")
  .bean(StateRequestMapper.class)   // maps locRequest → stateReq
  .to("cxf://http://crp11x1host/crp11x1?serviceClass=CRP11X1_Service")
  .bean(StateResponseMapper.class)  // maps stateResp → next form
```

What the nouns mean

- **Contract (WSDL/XSD)** – an XML file that promises "if you POST *this* XML shape to *this* URL you will get back *that* XML shape".
- **Java model** – the POJOs that CXF generates from the XSD; they hold the data so Java code can manipulate it without hand-parsing XML.
- **CXF** – the library that turns the WSDL contract into a living HTTP endpoint (inbound) or a typed client (outbound).
- **Configure endpoints** – writing `application.properties` lines such as `loc.camel.cxf.client.crp11x1.address=http://crp11x1host/crp11x1` so the route above knows where to send the request.
- **Implement mappings** – writing a 3-line MapStruct interface like
  `@Mapper public interface StateRequestMapper {`
  `CRP11X1StateRequest toStateRequest(LOC3X1BLocationRequest src); }`
  which the build turns into Java code that copies `src.getLocationId()` into `dest.setStateCode()`.

In one sentence The BPEL "receive-then-invoke" becomes a Camel "from-then-to" route; the WSDL/XSD contract becomes CXF endpoints plus generated Java models; the BPEL assignment becomes a MapStruct mapper; and you configure the actual partner URL in a properties file instead of hard-coding it in XML. *How files connect*: the BPEL references partner

links bound to WSDL ports; each WSDL references XSD types; the public interface (LOC3X1B) maps to internal (LOC3X1M) via field-level assignments. Dependencies are **contract-first**: types and operations in WSDL/XSD guide all message shapes.

Translation at a glance (why and how) We translate BPEL intent to **Camel routes + CXF endpoints** and **MapStruct mappings**. The sequence:

1. **Discover contracts**: read WSDL/XSD to know operations and message types.
2. **Generate Java models**: use CXF/JAXB to create Java classes/stubs from WSDL/XSD.
3. **Parse BPEL**: derive an orchestration spec (who we call, when, and how faults bubble).
4. **Generate routes**: create Camel routes that mirror receive → map → invoke partners → aggregate reply.
5. **Configure endpoints**: CXF inbound consumer (PAYLOAD) and CXF partner clients (PAYLOAD), with properties for addresses/timeouts/TLS.
6. **Implement mappings**: MapStruct interfaces to convert LOC3X1B request types to LOC3X1M and partner formats; stubs where incomplete.
7. **Build and run**: Maven compiles; Spring Boot exposes inbound BasicService; scripts drive SOAP Ping and capture evidence.
8. **Validate**: unit/integration tests check headers (e.g., `CxfConstants.OPERATION_NAME`), route flow, and mapping presence; evidence logged.

This works because **Camel** implements enterprise integration patterns (routing, transformation, error handling), **CXF** binds Java code to SOAP contracts, and **MapStruct** generates safe, fast field mappings from types — together reproducing the BPEL mediation in maintainable Java.

Technologies in simple words

- **Java**: the programming language used for services, clients, routes, and mappers.
- **Maven**: the build tool; compiles code, runs tests, manages dependencies, and packages JARs.
- **Spring Boot**: a framework for running services; it starts the inbound CXF service (BasicService).
- **Apache CXF**: connects Java to SOAP; defines endpoints and clients, marshals/unmarshals XML.
- **Apache Camel**: the routing engine; defines the orchestration as code (receive, call partners, handle errors).
- **MapStruct**: auto-generates mapping code between Java types (from WSDL-derived classes).
- **Python/FastAPI**: small API used to trigger runs and collect evidence for builds/tests/logs.

Why this translation makes sense BPEL describes *intent* (how messages flow). Camel+Cxf+MapStruct describe *implementation* with the same contracts. Using PAYLOAD mode in CXF keeps XML processing efficient and flexible while we incrementally add typed mappers. The outcome is **transparent, testable**, and **contract-aligned**.

Limitations, expectations, realities

- **Mappings are partial**: some MapStruct mappers are stubs; PAYLOAD routes still work but types may be `Object` until mappings grow.
- **Partner endpoints**: addresses/timeouts can be placeholders; live invocation may be blocked in tests without real endpoints.
- **Faults**: SimpleFault mapping is planned; current placeholder serialises JSON and should be replaced by schema-conformant mapping.
- **TLS**: optional; configure keystores and truststores in `application.properties` for secure partners.

Expectation: inbound service runs with SOAP Ping; routes set correct operation headers; tests and evidence report status; partner calls can be stubbed.

What is needed for success

- A working JDK + Maven; scripts runnable in the virtual environment (`venv/`).
- Valid WSDL/XSDs under `sample/`; properties set in `modules/*/src/main/resources/application.pro`
- Free local ports and correct CXF servlet mappings; optional TLS configured when required.
- Incremental completion of MapStruct mappers for business-critical fields.

What is the output and where

- Java sources and compiled JARs for modules (contracts, inbound, orchestration, clients, mappings).
- Generated contract code under `modules/contracts/target/generated-sources/*`.
- Evidence: logs and summaries under `tests/results/` and per-run copies under `uploads/<run_id>/resu`

Outputs are organised by module and by test run so you can trace failures and artefacts.

How it is validated

- **Unit tests**: mapper interfaces, header settings, small route fragments.
- **Integration tests**: end-to-end route execution with partner calls marked blocked when endpoints are unavailable.
- **Runtime evidence**: SOAP Ping request/response files, inbound logs, client logs.
- **Codegen logs**: CXF/JAXB output confirming contract coverage.

Confidence: high for inbound service bring-up and route wiring; medium for mapping completeness until stubs are replaced. Users should review outputs for critical fields and configure partners before relying on live calls.

Where results are saved

- Primary evidence: `tests/results/logs/*.log`, `tests/results/html/summary.txt`, mapper and unit test outputs.
- Generated contracts codegen: `modules/contracts/target/generated-sources/*` with preview logs at `tests/results/preflight/contracts-codegen.log`.
- Client and inbound evidence: `tests/results/logs/inbound-service.out`, `tests/results/logs/soap-ping.xml`, `tests/results/logs/soap-ping.response.xml`, and client meta under `tests/results/artifacts/clients/`.
- Per-run copies when triggered via API: `uploads/<run_id>/results/`.

9 Disection step-by-step

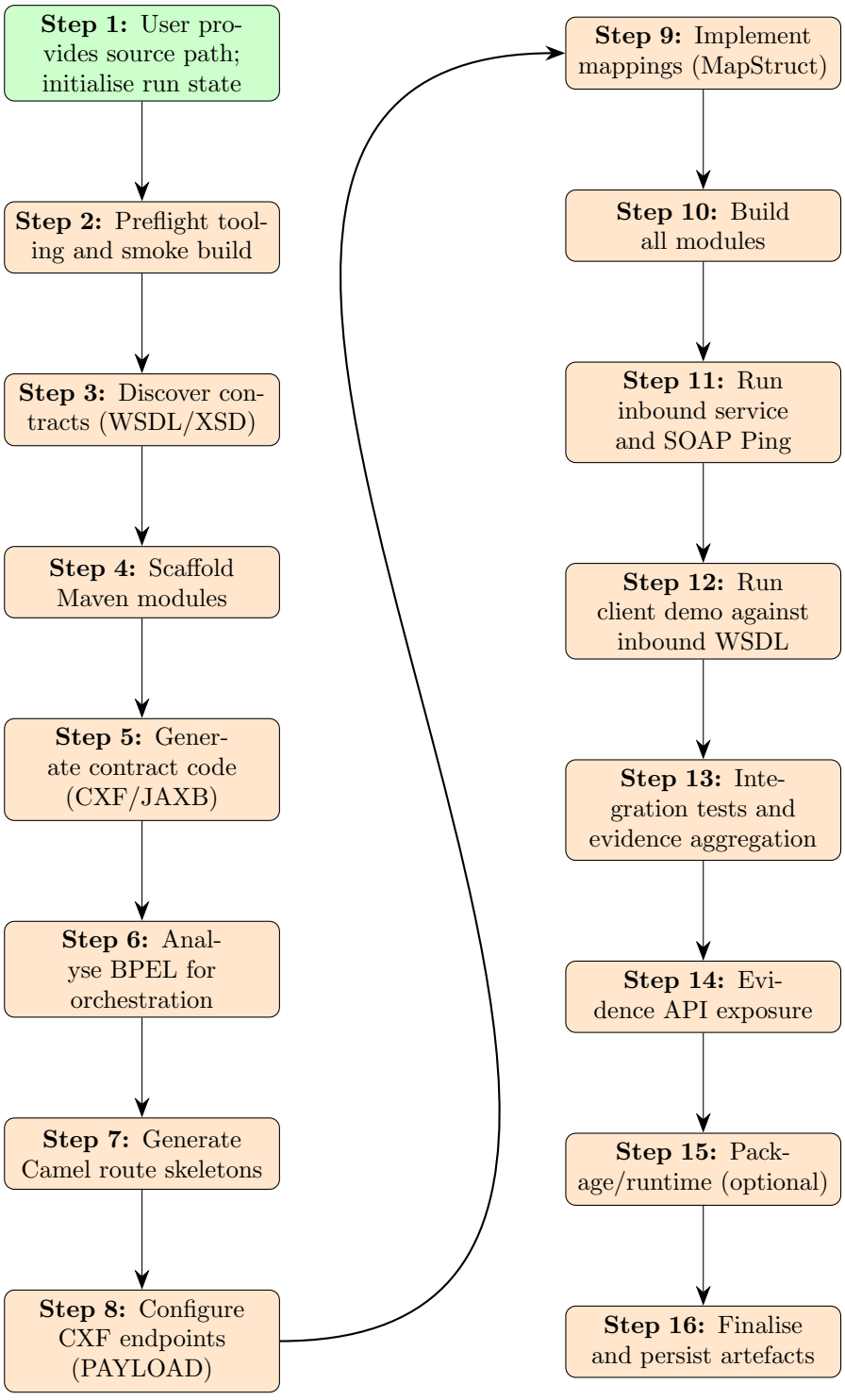

Figure 7: End-to-End Project Workflow: Steps 1–8 (left) and 9–16 (right).

Algorithm 1 End-to-End Project Workflow (Steps 1–8)

1: **Step 1:** User provides source path; initialise run state
   Inputs: ZIP upload or filesystem path
   Outputs: Run ID, prepared source directory
   Agent/Code: FastAPI `server/main.py` (`/api/run`)
2: **Step 2:** Preflight tooling and smoke build
   Inputs: System environment, `scripts/preflight.sh`
   Outputs: Tooling report, preflight logs
   Agent/Code: Preflight Agent (`tests/run_tests.py --preflight`)
3: **Step 3:** Discover contracts (WSDL/XSD)
   Inputs: `contracts/inventory.json`, `contracts/inventory.md`
   Outputs: Contract inventory, canonical map
   Agent/Code: `scripts/contracts_validator.py`, `contracts/canonical-map.yaml`
4: **Step 4:** Scaffold Maven modules
   Inputs: Repository structure
   Outputs: Updated parent `modules/pom.xml`
   Agent/Code: `scripts/scaffold_maven.py`
5: **Step 5:** Generate contract code (CXF/JAXB)
   Inputs: `sample/` WSDL/XSD
   Outputs: Generated sources, codegen log
   Agent/Code: `scripts/generate_contracts.sh`, Maven `generate-sources`
6: **Step 6:** Analyse BPEL for orchestration
   Inputs: `sample/LocationRetrievalLOC3X1/...bpel`
   Outputs: `orchestration/spec.yaml`
   Agent/Code: `scripts/analyze_bpel.py`, BPEL Agent
7: **Step 7:** Generate Camel route skeletons
   Inputs: Orchestration spec
   Outputs: `modules/orchestration/src/...Route.java`
   Agent/Code: `scripts/generate_routes.py`
8: **Step 8:** Configure CXF endpoints (PAYLOAD)
   Inputs: `application.properties`
   Outputs: CXF beans (`locationretrievalloc3x1bEndpoint`, partner clients)
   Agent/Code: `CxfServiceConfig.java`, `modules/inbound/CxfConfig.java`

Fine-Grained Dissection of Steps 1–16

Step 1: Initialise Run State and Source
What it does: Accepts a source tree or ZIP upload, prepares a working directory, and seeds pipeline metadata.
Agent/Script: FastAPI (`server/main.py`) via `/api/run`, optional CLI `start.py`.
Inputs: Uploaded ZIP or existing repo under `PROJECT_ROOT`; env `ENABLE_PIPELINE_RUNS`.
Process: Creates a run ID (UUID), sets up `uploads/<run_id>`, records initial metadata in in-memory `RUNS`.
Outputs: Run metadata (state, progress); upon completion, copies evidence to `uploads/<run_id>/results`.
Technology: FastAPI, Python threads, filesystem IO.

Step 2: Preflight Tooling and Smoke Build
What it does: Verifies JDK/Maven and required CLIs; can attempt a smoke build.
Agent/Script: `scripts/preflight_agent.py` (agentic checks), `scripts/preflight.sh` (shell preflight), and `tests/run_tests.py --preflight`.
Inputs: Local environment; project POMs; optional virtualenv `.venv`.

Process: Agent writes a concise report and optionally the shell preflight performs a smoke Maven build.
Outputs: `results/preflight/agent-checks.md`; when using shell preflight also `results/preflight/smoke.` and `results/reports/tooling-report.md`.
Technology: Python (Fast checks), Bash (smoke build), Maven.

Step 3: Discover and Validate Contracts
What it does: Scans WSDL/XSD/BPEL to build a canonical inventory and validates structure/consistency.
Agent/Script: `scripts/generate_contract_inventory.py`, `scripts/contracts_validator.py`; orchestrated by `scripts/agents/run.py` (`stage_contracts`).
Inputs: `sample/` WSDL/XSD/BPEL files.
Process: Generates `contracts/inventory.json` and `contracts/inventory.md`; cross-checks bindings, imports, namespaces against files.
Outputs: `contracts/inventory.json`, `contracts/inventory.md`, `contracts/canonical-map.yaml`; validator report `results/preflight/contracts-checks.md`.
Technology: Python, lxml/xml parsing, filesystem traversal.

Step 4: Scaffold Maven Modules
What it does: Ensures parent and client submodules are listed and structured correctly.
Agent/Script: `scripts/scaffold_maven.py`.
Inputs: `modules/` directory layout (`contracts`, `inbound`, `clients/*`, `mappings`, `orchestration`).
Process: Discovers `pom.xml` files and updates `modules/pom.xml` module list accordingly.
Outputs: Corrected parent POM with client submodules (e.g., `clients/locationretrievalloc3x1mpartner`
Technology: Python, XML/POM manipulation, filesystem.

Step 5: Generate Contract Code (CXF/JAXB)
What it does: Generates Java model classes and service stubs from WSDL/XSD.
Agent/Script: `scripts/generate_contracts.sh`; Maven `generate-sources`.
Inputs: `sample/` contracts; CXF/JAXB plugin configs.
Process: Invokes CXF/JAXB to emit classes under `modules/contracts/target/generated-sources/*`.
Outputs: Generated sources; codegen logs (e.g., `results/preflight/contracts-codegen.log`).
Technology: Apache CXF, JAXB, Maven plugins.

Step 6: Analyse BPEL for Orchestration
What it does: Interprets BPEL partner links and operations to derive mediation steps.
Agent/Script: `agents/bpel_agent.py`, `scripts/analyze_bpel.py`.
Inputs: BPEL files under `sample/LocationRetrievalLOC3X1/`.
Process: Extracts partner links, operations, and message shapes; emits `orchestration/spec.yaml`.
Outputs: `orchestration/spec.yaml` capturing flow and invocations.
Technology: Python, XML parsing, domain mapping.

Step 7: Generate Camel Route Skeletons
What it does: Produces Apache Camel routes aligned with the orchestration spec.
Agent/Script: `scripts/generate_routes.py`.
Inputs: `orchestration/spec.yaml`, contract inventory.
Process: Generates Java DSL routes (e.g., `GetLocationList3X1BRoute.java`) and documentation (`modules/orchestration/ROUTES.md`); optional YAML routes for Gradle subproject (`loc3x1-camel-route/src/main/resources/loc3x1-routes.yaml`).
Outputs: Route classes and mapping stubs (`direct:map*` routes).
Technology: Apache Camel (Java DSL and YAML), Spring Boot integration.

Step 8: Configure CXF Endpoints (PAYLOAD)
What it does: Wires inbound CXF consumer and partner CXF clients in PAYLOAD mode.
Agent/Script: `modules/orchestration/.../CxfServiceConfig.java`, `modules/inbound/.../CxfConfig`
Inputs: `modules/orchestration/src/main/resources/application.properties` for addresses and timeouts; optional TLS envs.
Process: Defines beans like `locationretrievalloc3x1bEndpoint` and partner clients (`stateorprovinceretr` etc.), sets `DataFormat=PAYLOAD`, `serviceClass=Object.class`, applies `receiveTimeout`/`connectionTimeout`
Outputs: Active CXF endpoints in the Camel context; inbound CXF servlet published at `/services/*` with `/BasicService`.
Technology: Apache CXF, Spring Boot auto-config (servlet), Camel CXF component.

Step 9: Implement Mappings (MapStruct)
What it does: Defines transformation mappers between request/reply schemas and internal DTOs.
Agent/Script: MapStruct interfaces under `modules/mappings/src/main/java/com/ei/camel/*MapMapper.` summary in `MAPPINGS.md`.
Inputs: Schema-derived classes (from Codegen) and domain DTOs.
Process: Author `@Mapper` interfaces with methods like `Object map(Object source)` (stubs) and targeted field mappings; add unit tests under `modules/mappings/src/test`.
Outputs: Compiled mapper implementations via MapStruct; tests verifying basic mapping expectations.
Technology: MapStruct, JUnit, Maven Surefire.

Step 10: Build All Modules
What it does: Compiles and packages contracts, inbound, orchestration, clients, mappings.
Agent/Script: `scripts/build_all.sh`; Maven multi-module build via `modules/pom.xml`.
Inputs: Source code and POM hierarchy.
Process: Runs `mvn clean package` across modules; emits JARs under `modules/*/target`.
Outputs: Build artefacts and test reports under `modules/*/target/surefire-reports`.
Technology: Maven, Java 17.

Step 11: Run Inbound Service and SOAP Ping
What it does: Launches Spring Boot + CXF inbound service and probes WSDL + SOAP Ping.
Agent/Script: `scripts/run_services.sh`; also exposed via `scripts/agents/run.py` (`stage_services`).
Inputs: Inbound JAR (`modules/inbound/target/inbound-*.jar`); env `INBOUND_PORT`.
Process: Starts the service, waits for WSDL at `/services/BasicService?wsdl`, posts a SOAP `Ping` request via `curl`.
Outputs: `tests/results/logs/inbound-service.out`, `soap-ping.xml`, `soap-ping.response.xml`; log summary.
Technology: Spring Boot, CXF servlet, curl.

Step 12: Run Client Demo Against Inbound WSDL
What it does: Builds and runs a CXF client that invokes `BasicService.Ping`.
Agent/Script: `scripts/run_clients.sh`; CXF runner `com.ei.clients.BasicServiceSoapClientRunner`.
Inputs: WSDL URL `BASIC_SERVICE_WSDL`, timeouts `CLIENT_CONNECT_TIMEOUT_MS`/`CLIENT_RECEIVE_TIMEOUT_`
Process: Builds `modules/clients/locationretrievalloc3x1b`, ensures inbound service is running, executes `mvn exec:java` with env overrides.
Outputs: `tests/results/logs/client-run.out`, and meta files under `tests/results/artifacts/clients/` (request parameters and extracted `Ping reply`).
Technology: Apache CXF dynamic client, Maven Exec Plugin.

Step 13: Integration Tests and Evidence Aggregation
What it does: Runs integration/unit tests, collects logs and HTML summary.
Agent/Script: `scripts/test_integrations.sh` (requires `.venv`); Python `tests/run_tests.py --all`.
Inputs: Test modules (`tests/test_*.py`); optional ordinal map `tests/ordinal-map.txt`.
Process: Discovers tests, executes with verbosity; writes logs to `tests/results/logs/integration-tests.lo` creates `tests/results/junit` and `tests/results/html`.
Outputs: JUnit XMLs (when configured), `summary.txt`, and aggregated logs under `tests/results/`.
Technology: Python unittest, file-based reporting.

Step 14: Evidence API Exposure
What it does: Serves health/status, evidence summaries, logs, and orchestrates pipeline runs.
Agent/Script: FastAPI (`server/main.py`); helper `scripts/start_api.sh`.
Inputs: `tests/results/` directory; env `ENABLE_PIPELINE_RUNS`.
Process: Exposes `/api/summary`, `/api/logs`, `/api/logs/{name}`, and run endpoints `/api/run`, `/api/runs`, `/api/run/{id}/status`, copying run outputs under `uploads/<run_id>/results`.
Outputs: JSON responses containing file tails and summary content; run metadata with metrics.
Technology: FastAPI, Uvicorn, CORS, simple run tracking.

Step 15: Packaging and Runtime (Optional)
What it does: Builds containers for inbound CXF service, Evidence API, and frontend; composes them locally.
Agent/Script: `docker-compose.yml`; `docker/Dockerfile.inbound` or `docker/Dockerfile.inbound.runti` `docker/Dockerfile.api`, `docker/Dockerfile.frontend`.
Inputs: Built JARs (host-built or in-image), API server code, frontend bundle; env for addresses/timeouts.
Process: Builds images and starts services; healthchecks hit `/services/BasicService?wsdl` and `/health`; frontend proxies `/api/*` to `api:8001`.
Outputs: Running containers: `camel-route-inbound`, `camel-route-api`, `camel-route-frontend`; exposed ports `8085`, `8001`, `8080`.
Technology: Docker, Compose, Nginx for static frontend and API proxy.

Step 16: Finalise and Persist Artefacts
What it does: Consolidates outputs per module and per run, ensuring traceability.
Agent/Script: FastAPI run pipeline; `scripts/agents/run.py` stages (env, plan, contracts, schema, scaffold, bpel, build, services, clients, tests).
Inputs: All prior stage outputs.
Process: Writes canonical evidence to `tests/results/`; mirrors into `uploads/<run_id>/results` for API-served runs; updates run status and metrics.
Outputs: Deterministic directory layout with logs, summaries, artefacts, and JARs.
Technology: Python orchestration, filesystem IO, Spring Boot/CXF runtime artefacts.

Algorithm 2 End-to-End Project Workflow (Steps 9–16)

1: **Step 9:** Implement mappings (MapStruct)
   Inputs: DTOs, schema-derived types
   Outputs: Mapper interfaces and unit tests
   Agent/Code: `modules/mappings/src/...MapMapper.java`, `...MapperTest.java`
2: **Step 10:** Build all modules
   Inputs: Maven project
   Outputs: Build artefacts, test reports
   Agent/Code: `scripts/build_all.sh`, Maven
3: **Step 11:** Run inbound service and SOAP Ping
   Inputs: Inbound module JAR
   Outputs: Runtime logs, ping request/response files
   Agent/Code: `scripts/run_services.sh`
4: **Step 12:** Run client demo against inbound WSDL
   Inputs: WSDL URL, timeouts
   Outputs: Client run log, ping meta
   Agent/Code: `scripts/run_clients.sh`
5: **Step 13:** Integration tests and evidence aggregation
   Inputs: `tests/run_tests.py`
   Outputs: JUnit XML, `tests/results/html/summary.txt`
   Agent/Code: Master Test Runner (`scripts/test_master.sh`)
6: **Step 14:** Evidence API exposure
   Inputs: Test outputs in `tests/results/*`
   Outputs: `/api/summary`, `/api/logs/*`, `/api/run/*`
   Agent/Code: FastAPI `server/main.py`
7: **Step 15:** Package/runtime (optional)
   Inputs: Dockerfiles, compose
   Outputs: Containers, local endpoints
   Agent/Code: `docker/*`, `docker-compose.yml`
8: **Step 16:** Finalise and persist artefacts
   Inputs: All previous outputs
   Outputs: Results under `tests/results/`; per-run copy under `uploads/<run_id>/results/`
   Agent/Code: Pipeline orchestrated by API and scripts

10 Conclusion

The transformation is feasible and valuable. With disciplined agentic automation and strict adherence to the original contracts and BPEL intent, the resulting project will be maintainable, verifiable, and aligned with modern Java integration practices.